Файлы CSV

Теория

```
date;film;price;sold
01.10;Лето;350,5;42
02.10;"Он; она";280;9
03.10;Свет;300;31
```



Только то, что нужно к задаче на таблицу на школьном этапе. Пример — файл `films.csv`, для времени — `log.csv`; в конце — задачи прошлых лет и ещё две на группы и время, с разбором. Код идёт подряд, как ячейки ноутбука: что объявлено выше, тем пользуются ниже. Под кодом, с чертой слева, — ровно то, что он печатает.

Что такое CSV

Таблица, записанная текстом: строка файла — строка таблицы, значения через запятую, первая строка — названия столбцов. Файл `films.csv`:

```
title,genre,year,rating,views
Северный ветер,драма,1998,7.4,120
Город,комедия,2004,6.9,340
Лето, Драма,2011,8.0,95
Мост,триллер,2004,7.1,210
Остров,драма,2006,,60
Свет,комедия,2015,н/д,410
Река,триллер,2019,7.8,150
Сад,драма,2021,6.5,80
```

В нём две неаккуратности, обычные для олимпиадных таблиц: у «Острова» оценки нет, у «Света» вместо неё `н/д`, а у «Лета» жанр с пробелом и заглавной буквой.

Чтение

```
import pandas as pd
df = pd.read_csv("films.csv")
print(df)
print(df.shape)
```

```
             title    genre  year rating  views
0  Северный ветер    драма  1998    7.4    120
1           Город  комедия  2004    6.9    340
2            Лето    Драма  2011    8.0     95
3            Мост  триллер  2004    7.1    210
4          Остров    драма  2006    NaN     60
5            Свет  комедия  2015    н/д    410
6            Река  триллер  2019    7.8    150
7             Сад    драма  2021    6.5     80
(8, 5)
```

Слева — номера строк, `df.shape` — сколько строк и столбцов. Длинную таблицу целиком не печатают: `df.head()` — первые пять строк.

Если файл прочитался не так:

- всё склеилось в один столбец — разделитель не запятая: `pd.read_csv("f.csv", sep=";")`;
- числа записаны с запятой, `350,5`, — добавить `decimal=","`;
- ошибка `UnicodeDecodeError` — другая кодировка: `encoding="cp1251"`.

Чистка

Пустое поле pandas читает как пропуск `NaN`, а `н/д` — как текст, и тогда текстом становится весь столбец: числа в нём уже не посчитать. Лишний пробел и заглавная буква делают из драмы ещё один жанр:

```
print(df["rating"].tolist())
print(df["genre"].value_counts())
```

```
['7.4', '6.9', '8.0', '7.1', nan, 'н/д', '7.8', '6.5']
genre
драма      3
комедия    2
триллер    2
 Драма     1
Name: count, dtype: int64
```

Чинится двумя строками:

```
df["rating"] = pd.to_numeric(df["rating"],
                             errors="coerce")
df["genre"] = df["genre"].str.strip().str.lower()
print(df["rating"].tolist())
print(df["genre"].value_counts())
```

```
[7.4, 6.9, 8.0, 7.1, nan, nan, 7.8, 6.5]
genre
драма      4
комедия    2
триллер    2
Name: count, dtype: int64
```

`to_numeric(..., errors="coerce")` делает пропуском всё, что не число. `str.strip()` убирает пробелы по краям, `str.lower()` переводит в строчные. Что ещё бывает нужно:

- `df["rating"].isna().sum()` — сколько пропусков;

- `df = df.dropna(subset=["rating"])` — выбросить строки с пропуском;
- `df["rating"] = df["rating"].fillna(0)` — заполнить пропуски;
- `df = df.drop_duplicates()` — убрать повторы строк.

Такие команды таблицу не меняют, а возвращают новую — поэтому результат присваивают обратно.

Выбор строк

Условие — в квадратных скобках после таблицы. Каждое сравнение — в круглых, «и» — `&`, «или» — `|`, «не» — `~`:

```
d = df[(df["genre"] == "драма")
       & (df["year"] >= 2000)]
print(d)
print(len(d))
```

```
    title  genre  year  rating  views
2    Лето  драма  2011     8.0     95
4  Остров  драма  2006     NaN     60
7     Сад  драма  2021     6.5     80
3
```

`len(d)` — сколько строк подошло. Ещё:

- `df[df["year"].between(2000, 2010)]` — оба конца входят; если один не входит — двумя сравнениями;
- `df[df["genre"].isin(["драма", "триллер"])]` — одно из значений;
- `df[df["rating"].isna()]` — строки с пропуском.

Счёт по столбцу

```
v = df["views"]
print(v.sum(), v.mean(), v.median(), v.max())
print(df["rating"].mean(), df["rating"].count())
print(df.loc[v.idxmax(), "title"])
```

```
1465 183.125 135.0 410
7.283333333333334 6
Свет
```

Средняя оценка — по шести фильмам, а не по восьми: пропуски не считаются, `count()` их тоже не включает. `idxmax()` — номер строки с наибольшим значением, `df.loc[номер, "столбец"]` — что стоит в этой строке. Ещё `min()` и `idxmin()`, `nunique()` — сколько различных значений, `mode()[0]` — самое частое.

Новый столбец считают формулой из старых, сразу по всем строкам:

```
df["age"] = 2026 - df["year"]
print(df[["title", "year", "age"]].head(3))
```

```
             title  year  age
0  Северный ветер  1998   28
1           Город  2004   22
2            Лето  2011   15
```

Группы и сортировка

`groupby` — то же, что сводная таблица в Excel: строки делятся по значению столбца, и счёт идёт по каждой группе отдельно.

```
g = df.groupby("genre")["views"].sum()
print(g)
print(g.idxmax(), g.max())
print(df.groupby("genre")["rating"].mean().round(2))
```

```
genre
драма      355
комедия    750
триллер    360
Name: views, dtype: int64
комедия 750
genre
драма      7.30
комедия    6.90
триллер    7.45
Name: rating, dtype: float64
```

Вместо `sum` подходит всё из счёта по столбцу; `df.groupby("genre").size()` — сколько строк в каждой группе. Результат — снова столбец, и выбор строк к нему применяется так же: `g[g > 500]` — группы, где больше 500.

```
top = df.sort_values("views", ascending=False)
print(top[["title", "views"]].head(3))
```

```
   title  views
5   Свет    410
1  Город    340
3   Мост    210
```

`ascending=False` — по убыванию, без него — по возрастанию.

Значение группы в каждой строке

`groupby(...).sum()` сворачивает группу в одно число. Когда строку надо сравнить с её группой, итог группы нужен рядом с каждой строкой. Это `transform`: он возвращает столбец той же длины, что таблица, и в каждой строке стоит итог её группы.

```
df["genre_views"] = (df.groupby("genre")["views"]
                     .transform("sum"))
df["share"] = (df["views"] / df["genre_views"]).round(2)
print(df[["title", "genre", "views",
          "genre_views", "share"]])
```

```
             title    genre  views  genre_views  share
0  Северный ветер    драма    120          355   0.34
1           Город  комедия    340          750   0.45
2            Лето    драма     95          355   0.27
3            Мост  триллер    210          360   0.58
4          Остров    драма     60          355   0.17
5            Свет  комедия    410          750   0.55
6            Река  триллер    150          360   0.42
7             Сад    драма     80          355   0.23
```

Доля фильма в просмотрах своего жанра — деление двух столбцов, строка на строку. С `sum()` так не выйдет: там три числа, а строк восемь.

Сравнить каждую строку со средним своей группы:

```
mean_r = df.groupby("genre")["rating"].transform("mean")
print(mean_r.round(2).tolist())
print(df.loc[df["rating"] > mean_r, "title"].tolist())
```

```
[7.3, 6.9, 7.3, 7.45, 7.3, 6.9, 7.45, 7.3]
['Северный ветер', 'Лето', 'Река']
```

Пропуск у «Острова» в среднее драм не вошёл, а сравнение с пропуском даёт `False`: такая строка не попадает ни в

«больше», ни в «меньше». `df.loc[условие, "столбец"]` — строки по условию и один столбец.

Лучшие в своей группе:

```
best = df.groupby("genre")["views"].transform("max")
print(df.loc[df["views"] == best,
             ["genre", "title", "views"]])
```

```
     genre           title  views
0    драма  Северный ветер    120
3  триллер            Мост    210
5  комедия            Свет    410
```

Если максимум делят двое, попадут оба — в отличие от `idxmax`, который берёт первого.

В `transform` годится то же, что в группировке: `"sum"`, `"mean"`, `"median"`, `"max"`, `"min"`, `"count"`, `"size"`. Ещё два частых случая:

- заполнить пропуск средним своей группы:

  ```
  r = df["rating"].fillna(
      df.groupby("genre")["rating"].transform("mean"))
  print(r.round(2).tolist())
  ```

  ```
  [7.4, 6.9, 8.0, 7.1, 7.3, 6.9, 7.8, 6.5]
  ```

  Здесь результат — в отдельном `r`, чтобы не менять таблицу ниже; в задаче его присваивают обратно в столбец;

- номер строки внутри группы, с нуля — `df.groupby("g").cumcount()`.

Ответ

Округлять — самому. `round` в Python округляет половину к чётному, а не вверх, как в школе:

```
print(round(2.5), round(3.5), round(0.125, 2))
print(f"{df['rating'].mean():.4f}")
```

```
2 4 0.12
7.2833
```

Поэтому число печатают с запасом знаков, как во второй строке, и округляют по условию руками.

Ответ файлом — без номеров строк, `index=False`:

```
res = g.reset_index()
res.to_csv("answer.csv", index=False)
print(open("answer.csv", encoding="utf-8").read())
```

```
genre,views
драма,355
комедия,750
триллер,360
```

`reset_index()` превращает результат группировки обратно в таблицу из двух столбцов. Названия и порядок столбцов — как в условии.

Даты и время

Столбец со временем читают как время, а не как текст: иначе его не сравнить и не вычесть. Файл `log.csv` — журнал действий на сайте:

```
user,time,action
anna,2026-10-19 09:58,вход
boris,2026-10-19 10:05,вход
anna,2026-10-19 10:20,задача
anna,2026-10-19 11:30,задача
boris,2026-10-19 10:40,задача
anna,2026-10-20 08:15,вход
boris,2026-10-20 23:50,задача
anna,2026-10-20 08:40,задача
```

```
log = pd.read_csv("log.csv", parse_dates=["time"])
print(log.dtypes)
```

```
user                 str
time      datetime64[us]
action               str
dtype: object
```

`parse_dates` делает столбец временем, `datetime64`; без него он остался бы строкой. Строки файла не обязаны идти по времени — сначала сортировка:

```
log = log.sort_values(["user", "time"])
print(log)
```

```
    user                time  action
0   anna 2026-10-19 09:58:00    вход
2   anna 2026-10-19 10:20:00  задача
3   anna 2026-10-19 11:30:00  задача
5   anna 2026-10-20 08:15:00    вход
7   anna 2026-10-20 08:40:00  задача
```

```
1  boris 2026-10-19 10:05:00    вход
4  boris 2026-10-19 10:40:00  задача
6  boris 2026-10-20 23:50:00  задача
```

Сортировка по двум столбцам: сначала по пользователю, внутри — по времени.

Def (части даты). Через `.dt`:

```
t = log["time"]
print(t.dt.date.iloc[0])
print(t.dt.hour.tolist())
print(t.dt.dayofweek.tolist())
```

```
2026-10-19
[9, 10, 11, 8, 8, 10, 10, 23]
[0, 0, 0, 1, 1, 0, 0, 1]
```

`dt.date` — дата без времени, `dt.hour` — час, `dt.dayofweek` — день недели, понедельник — 0. Ещё `dt.month`, `dt.minute`, `dt.floor("h")` — момент, округлённый вниз до часа.

Def (сравнение и разность). Время сравнивают со строкой-датой, а разность двух моментов — промежуток, `Timedelta`:

```
late = log[log["time"] >= "2026-10-20"]
print(late["action"].tolist())
d = t.max() - t.min()
print(d, d.total_seconds() / 3600)
```

```
['вход', 'задача', 'задача']
1 days 13:52:00 37.86666666666667
```

`"2026-10-20"` — полночь 20 октября, так что «не раньше» берёт весь этот день. `total_seconds()` переводит промежуток в секунды, дальше — в часы или минуты делением.

Def (счёт по дням). Группировка по дате:

```
print(log.groupby(log["time"].dt.date).size())
```

```
time
2026-10-19    5
2026-10-20    3
dtype: int64
```

Так же по часу — `dt.hour`, по дню недели — `dt.dayofweek`.

Def (промежуток до предыдущей строки). `diff()` — разность с предыдущей строкой; внутри группы — отдельно для каждого пользователя:

```
log["gap"] = log.groupby("user")["time"].diff()
print(log[["user", "time", "gap"]])
```

```
    user                time             gap
0   anna 2026-10-19 09:58:00             NaT
2   anna 2026-10-19 10:20:00 0 days 00:22:00
3   anna 2026-10-19 11:30:00 0 days 01:10:00
5   anna 2026-10-20 08:15:00 0 days 20:45:00
7   anna 2026-10-20 08:40:00 0 days 00:25:00
1  boris 2026-10-19 10:05:00             NaT
4  boris 2026-10-19 10:40:00 0 days 00:35:00
6  boris 2026-10-20 23:50:00 1 days 13:10:00
```

У первого действия пользователя предыдущего нет — там `NaT`, пропуск для времени. Без `groupby("user")` разность посчиталась бы и между последним действием Анны и первым Бориса. И без сортировки выше `diff` сравнил бы строки в порядке файла.

Def (другой формат и свои промежутки). Дату «по-русски» pandas сам не разберёт — формат задают явно:

```
s = pd.Series(["19.10.2026 9:05", "20.10.2026 23:50"])
print(pd.to_datetime(s, format="%d.%m.%Y %H:%M"))
print(pd.Timedelta(minutes=90),
      pd.Timestamp("2026-10-19") + pd.Timedelta(days=3))
```

```
0   2026-10-19 09:05:00
1   2026-10-20 23:50:00
dtype: datetime64[us]
0 days 01:30:00 2026-10-22 00:00:00
```

`%d` — день, `%m` — месяц, `%Y` — год четырьмя цифрами, `%H:%M` — часы и минуты. Строки, которые под формат не подошли, `errors="coerce"` сделает пропусками. Сетка моментов с постоянным шагом — `pd.date_range(начало, конец, freq="10min")`; шаги `"min"`, `"h"`, `"D"`.

Школьный этап 2025

Задача на таблицу — шестая из восьми. Файл дают в XLSX, ODS или CSV на выбор; проще всего взять CSV. Настоящие таблицы есть только в тренажёре Сириуса, поэтому у каждой задачи ниже свой маленький файл: на нём виден вывод. Условия пересказаны коротко.

Группа 1 (Участки). Строка файла — участок: x_1 клёнов, x_2 лип и отметка y, подходит ли он, 1 или 0. Оценка участка $s = 0{,}4x_1 + 0{,}6x_2$. Сколько участков с $y = 1$ и оценкой не ниже медианы всех оценок? Файл `uchastki.csv`:

```
x1,x2,y
5,1,1
0,4,1
3,2,1
1,0,0
7,3,0
```

```
df = pd.read_csv("uchastki.csv")
df["s"] = 0.4 * df["x1"] + 0.6 * df["x2"]
m = df["s"].median()
print(df["s"].tolist())
print(m)
print(len(df[(df["y"] == 1) & (df["s"] >= m)]))
```

```
[2.6, 2.4, 2.4000000000000004, 0.4, 4.6]
2.4000000000000004
2
```

Ответ здесь — 3: у участков $(0, 4)$ и $(3, 2)$ оценка одна и та же, 2,4, и она же медиана. Но 0,4 и 0,6 в двоичной записи не точны, и одно и то же 2,4, посчитанное разными путями, вышло двумя разными числами. Сравнение «не ниже» ломается как раз на равенстве. Лечится счётом в целых: оценка, умноженная на 10, точна.

```
df["s10"] = 4 * df["x1"] + 6 * df["x2"]
m10 = df["s10"].median()
print(df["s10"].tolist(), m10)
print(len(df[(df["y"] == 1) & (df["s10"] >= m10)]))
```

```
[26, 24, 24, 4, 46] 24.0
3
```

При чётном числе значений `median()` берёт полусумму двух средних — как и требует условие.

Группа 2 (Три толстяка). В строке — мнения трёх толстяков n_1, n_2, n_3 и решение наследника y, всё 0 или 1. Предсказание повара — 1, если единиц среди мнений хотя бы две. В скольких строках оно совпало с y?

```
df = pd.read_csv("tolstyaki.csv")
golosa = df[["n1", "n2", "n3"]].sum(axis=1)
df["pred"] = (golosa >= 2).astype(int)
print(df)
print((df["pred"] == df["y"]).sum())
```

```
   n1  n2  n3  y  pred
0   1   1   0  1     1
1   0   0   1  0     0
2   1   0   0  1     0
3   1   1   1  1     1
4   0   1   1  0     1
5   0   0   0  0     0
4
```

`sum(axis=1)` складывает по строке, без него — по столбцу. Сравнение даёт `True` и `False`, `astype(int)` делает из них 1 и 0. Сумма столбца из `True` и `False` — число `True`: так и считают «в скольких строках».

Группа 3 (Выбросы). В каждой строке пять натуральных чисел, x — третье. Значение — выброс, если $|x - \bar{x}| > \bar{x}$, где $\bar{x}$ — среднее третьего столбца. Сколько выбросов? Файл `vybrosy.csv`:

```
a,b,c,d,e
4,7,2,1,9
3,3,11,5,2
8,1,3,6,6
2,9,1,4,3
5,5,10,2,8
1,6,3,7,4
9,2,2,3,1
6,4,8,8,5
```

```
df = pd.read_csv("vybrosy.csv")
x = df.iloc[:, 2]
m = x.mean()
print(x.tolist(), m)
print(((x - m).abs() > m).sum())
```

```
[2, 11, 3, 1, 10, 3, 2, 8] 5.0
1
```

`iloc[:, 2]` — столбец по номеру, считая с нуля: его имя знать не нужно. Раз $x > 0$, неравенство $\bar{x} - x > \bar{x}$ невозможно, и выброс — просто $x > 2\bar{x}$: здесь $11 > 10$, а 10 уже нет, неравенство строгое.

Если первая строка файла — уже числа, а не названия, читать с `header=None`: иначе она станет заголовком и выпадет из счёта. Проверка — `df.shape`.

Группа 4 (Порог). В строке — вероятность p и верная метка y. Предсказание — 1 при $p \geq 0{,}6$, иначе 0. Сколько совпадений с y? Файл `porog.csv`:

```
p,y
0.9,1
0.6,1
0.55,1
0.3,0
0.7,0
0.6,0
```

```
df = pd.read_csv("porog.csv")
pred = (df["p"] >= 0.6).astype(int)
print(pred.tolist())
print((pred == df["y"]).sum())
```

```
[1, 1, 0, 0, 1, 1]
3
```

Две строки ровно на пороге. Число `0.6` из файла и `0.6` в коде — одна и та же запись и одно и то же число, поэтому `>=` их берёт. Подвох был бы, если бы p получалось вычислением, как оценка в задаче про участки.

Школьный этап 2026

Задача на таблицу — четвёртая из шести: чистка по шагам из условия и несколько вопросов подряд. Таблицы к ней не опубликованы, файлы ниже свои.

Группа 3 (Схожесть решений). Система сравнила посылки участников попарно: в строке — участники `student_1` и `student_2`, задача `task_id` и схожесть `similarity`. Сначала чистка: пустая схожесть — 0, меньше 0 — 0, больше 1 — 1, сравнения участника с самим собой выбросить. Строка подозрительна, если схожесть не меньше 0,92. Пара участников, без учёта порядка, нарушила правила, если у неё подозрительные строки по трём и более разным задачам. Сколько подозрительных строк? Сколько пар-нарушителей? Сколько в них участников? Файл `similarity.csv`:

```
student_1,student_2,task_id,similarity
17,42,1,0.95
42,17,2,1.3
17,42,2,0.97
17,42,3,0.93
5,5,1,0.99
5,8,1,
5,8,2,0.92
8,5,3,0.94
5,8,4,-0.2
8,9,1,0.96
8,9,2,0.99
9,8,4,0.92
```

```
df = pd.read_csv("similarity.csv")
s = df["similarity"].fillna(0)
df["similarity"] = s.clip(0, 1)
df = df[df["student_1"] != df["student_2"]]
sus = df[df["similarity"] >= 0.92].copy()
print(len(sus))
```

```
9
```

Шаги — в порядке условия: чистка раньше порога, иначе 1,3 не станет единицей и строка выпадет из счёта. `clip(0, 1)` обрезает с обеих сторон сразу. `.copy()` — потому что к выбранным строкам дальше добавляются столбцы: без копии pandas 2 предупреждает, что не знает, меняется ли при этом исходная таблица.

```
para = sus[["student_1", "student_2"]]
sus["a"] = para.min(axis=1)
sus["b"] = para.max(axis=1)
z = sus.groupby(["a", "b"])["task_id"].nunique()
nar = z[z >= 3]
print(nar)
print(len(nar))
```

```
a   b
8   9     3
17  42    3
Name: task_id, dtype: int64
2
```

Пары $(42, 17)$ и $(17, 42)$ — одна: в `a` идёт меньший номер, в `b` — больший. Группировка по двум столбцам даёт двойной индекс — саму пару. `nunique()` считает разные задачи: две подозрительные строки пары 17 и 42 по задаче 2 дают одну задачу. Пара 5 и 8 набрала только две.

```
r = nar.reset_index()
print(pd.concat([r["a"], r["b"]]).nunique())
```

```
4
```

Участник может быть в нескольких парах сразу. Поэтому номера из обоих столбцов ставятся в один, `concat`, и считаются различные.

Группа 4 (Чемпионат). 16 команд, каждая с каждой дома и в гостях; в строке — хозяева, гости и голы тех и других. Победа — 3 очка, ничья — 1. Кто первый, на сколько очков он опережает второго, кто последний? Затем последняя команда снимается, все матчи с ней аннулируются: сколько очков у нового победителя? Файл `matches.csv`, команд в нём четыре:

```
home_team,away_team,home_goals,away_goals
Сокол,Ракета,1,1
Ракета,Сокол,2,0
Сокол,Звезда,0,1
Звезда,Сокол,1,1
Сокол,Волна,3,0
Волна,Сокол,0,2
Ракета,Звезда,2,2
Звезда,Ракета,1,0
Ракета,Волна,3,0
Волна,Ракета,0,1
Звезда,Волна,1,1
Волна,Звезда,2,1
```

Матч даёт очки двум командам, а стоят они в разных столбцах. Поэтому каждый матч превращается в две строки: от лица хозяев и от лица гостей.

```
m = pd.read_csv("matches.csv")

def tablica(m):
    dom = m.rename(columns={"home_team": "team",
        "home_goals": "za", "away_goals": "pr"})
    gos = m.rename(columns={"away_team": "team",
        "away_goals": "za", "home_goals": "pr"})
    v = pd.concat([dom, gos])
    win = v["za"] > v["pr"]
    draw = v["za"] == v["pr"]
    v["pts"] = 3 * win + draw
    t = v.groupby("team")["pts"].sum()
```

```
    return t.sort_values(ascending=False)

t = tablica(m)
print(t)
print(t.index[0], t.iloc[0] - t.iloc[1])
print(t.index[-1])
```

```
team
Ракета    11
Звезда     9
Сокол      8
Волна      4
Name: pts, dtype: int64
Ракета 2
Волна
```

`rename` переименовывает столбцы так, что у обеих половин одни и те же `team`, `za`, `pr`, и `concat` ставит их друг под другом. В арифметике `True` — 1, `False` — 0, поэтому `3 * win + draw` — уже очки. `t.index[0]` — имя в первой строке, `t.iloc[0]` — число в ней. Счёт завёрнут в функцию: ниже он нужен второй раз.

```
last = t.index[-1]
ost = m[(m["home_team"] != last)
        & (m["away_team"] != last)]
print(tablica(ost))
```

```
team
Звезда    8
Ракета    5
Сокол     2
Name: pts, dtype: int64
```

Таблицу считают заново, а не правят старую: лидер сменился, потому что «Ракета» взяла шесть из одиннадцати очков на «Волне». При равных очках порядок после сортировки не определён — поэтому условие и обещает, что первое и последнее места однозначны.

Разбор: группы и время

Ещё две задачи того же рода, что на туре, но на приёмы из разделов «Значение группы в каждой строке» и «Даты и время». Файлы свои.

A (Крупные чеки). Чеки трёх магазинов: магазин, кассир, сумма. Сколько чеков больше удвоенного среднего чека своего магазина? В каком магазине самый крупный чек составляет наибольшую долю выручки? Файл `cheki.csv`:

```
shop,cashier,amount
Север,Аня,350
Север,Борис,1200
Север,Аня,410
Север,Вера,380
Юг,Глеб,900
Юг,Глеб,950
Юг,Дина,4100
Юг,Дина,870
Центр,Ева,2000
Центр,Жора,2100
```

Средний чек нужен свой рядом с каждым чеком — это `transform`:

```
ch = pd.read_csv("cheki.csv")
ch["shop_mean"] = (ch.groupby("shop")["amount"]
                   .transform("mean"))
print(ch)
print((ch["amount"] > 2 * ch["shop_mean"]).sum())
```

```
    shop cashier  amount  shop_mean
0  Север     Аня     350      585.0
1  Север   Борис    1200      585.0
2  Север     Аня     410      585.0
3  Север    Вера     380      585.0
4     Юг    Глеб     900     1705.0
```

```
5     Юг    Глеб     950     1705.0
6     Юг    Дина    4100     1705.0
7     Юг    Дина     870     1705.0
8  Центр     Ева    2000     2050.0
9  Центр    Жора    2100     2050.0
2
```

Подошли 1200 на «Севере», это больше $2 \cdot 585 = 1170$, и 4100 на «Юге». Ловушка — сравнить со средним по всем чекам: оно 1326, удвоенное — 2652, и чек остался бы один. Крупный чек сам входит в среднее своего магазина и поднимает его — условие так и говорит, пересчитывать без него не нужно.

Для второго вопроса — доля чека в выручке магазина и максимум по магазину:

```
ch["shop_sum"] = (ch.groupby("shop")["amount"]
                  .transform("sum"))
ch["share"] = ch["amount"] / ch["shop_sum"]
top = ch.groupby("shop")["share"].max()
print(top.round(3))
print(top.idxmax())
```

```
shop
Север    0.513
Центр    0.512
Юг       0.601
Name: share, dtype: float64
Юг
```

Здесь нужны оба вида группировки: `transform` даёт выручку магазина в каждой строке, чтобы поделить, а `max()` сворачивает магазин в одно число, чтобы сравнить магазины.

Б (Сессии). Журнал `log.csv` из раздела о времени. Сессия пользователя обрывается, если между соседними его действиями прошло больше 30 минут. Сколько всего сессий? Сколько минут длится самая длинная — от первого её действия до последнего?

Промежутки до предыдущего действия уже посчитаны — столбец `gap`. Новая сессия начинается там, где предыдущего действия нет или оно было больше получаса назад:

```
log = log.sort_values(["user", "time"])
new = (log["gap"].isna()
       | (log["gap"] > pd.Timedelta(minutes=30)))
log["session"] = new.cumsum()
print(log[["user", "time", "session"]])
print(log["session"].nunique())
```

```
    user                time  session
0   anna 2026-10-19 09:58:00        1
2   anna 2026-10-19 10:20:00        1
3   anna 2026-10-19 11:30:00        2
5   anna 2026-10-20 08:15:00        3
7   anna 2026-10-20 08:40:00        3
1  boris 2026-10-19 10:05:00        4
4  boris 2026-10-19 10:40:00        5
6  boris 2026-10-20 23:50:00        6
6
```

Главный приём: `cumsum()` от столбца из `True` и `False`. Каждое `True` — начало сессии — прибавляет единицу, и все строки одной сессии получают один номер. Борис разделился надвое: между его действиями 35 минут, это больше 30. Порядок строк здесь решает всё — поэтому сортировка ещё раз, хотя журнал уже отсортирован.

```
dur = log.groupby("session")["time"].agg(["min", "max"])
dur["minutes"] = ((dur["max"] - dur["min"])
                  .dt.total_seconds() / 60)
```

```
print(dur["minutes"])
print(dur["minutes"].max())
```

```
session
1    22.0
2     0.0
3    25.0
4     0.0
5     0.0
6     0.0
Name: minutes, dtype: float64
25.0
```

`agg(["min", "max"])` — два итога группы сразу, двумя столбцами. Сессия из одного действия длится 0 минут. У столбца промежутков, как и у столбца моментов, свой `.dt`: `dt.total_seconds()`.